CONSTRUCTION AND CAPACITY STUDY

A 10,000-query/s key–value database in ten-party honest-majority MPC

Updated 13 October 2026, America/New_York

Target: 2^{28} records, 32-byte keys and values; ten parties, at most four malicious corruptions, security with abort; average response time at most five seconds.

1 Decision for the proposal

Continue with two cuckoo tables over binary Circuit OPRAM, but make the arithmetic compilation and private routing the research contribution. Probe both tables in parallel; use a shared central batcher, grouped recursive-map updates, private dot products, a larger flat base, and a sweep of position-map packing. The important distinction is between increasing the number of labels per recursive record and changing the ORAM tree's branching factor: this study changes the former and retains the binary-tree eviction invariant.

A low annual budget is not yet supported. The explicit all-arithmetic reference used here has millions of secure reduction outputs per logical read. Its AWS sensitivity scenarios land in the billion-dollar/year range. That is a warning about this compilation, not an impossibility result for the problem. No complete malicious ten-party Circuit OPRAM benchmark or proof of these new compiled circuits was produced. In particular, the model does not establish the best achievable construction among mixed-domain, shuffle-based, hierarchical, or other MPC memory protocols.

The useful budget milestone is to measure and reduce the complete per-read work vector. A backend-conditional envelope below shows that even 10,000–30,000 secure outputs per read can consume millions of dollars/year across ten parties. Those counts are *optimization targets*, not achieved costs of the proposed system.

Suggested proposal language.

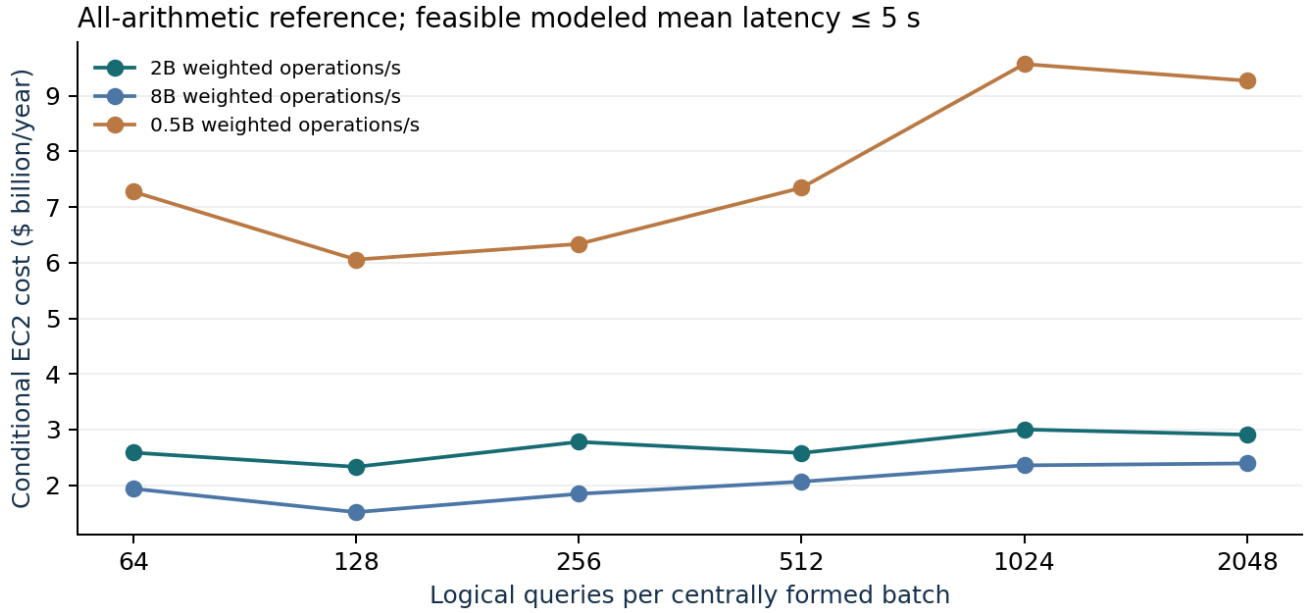
> We will develop an honest-majority MPC key–value service using batched oblivious access rather than a direct multiparty garbled-RAM extension. Our design combines parallel cuckoo probes, arithmetic-aware recursive position maps, private inner products, and optimized oblivious routing. We target 10,000 reads per second and mean response time below five seconds. Preliminary circuit and cloud accounting identifies metadata conversion and pool routing as the principal bottlenecks; meeting a practical cost target requires reducing these costs. We will validate throughput and annual operating costs with a complete malicious ten-party implementation including preprocessing, maintenance, and all-party communication.

2 What the numerical search found

The program searches batches 64–2048, fanouts 4–256, flat cutoffs 256–65536, compact versus field-bit metadata, sorting networks versus a checked-shuffle quicksort template, five EC2 instance sizes, and machines/replicas. It minimizes EC2 expenditure among points with modeled mean latency below five seconds. The cutoffs are requested limits; the actual flat map may be larger when another recursive tree would be too small to support its parallel batch.

Scenario	Batch	Fanout	Actual base	$/year (B)	Mean (s)
central	128	128	32,768	2.33	4.77
faster_cpu	128	128	32,768	1.52	4.65
slower_cpu	128	128	32,768	6.06	2.92
low_exchange	512	32	16,384	1.69	2.61
slow_exchange	64	128	32,768	4.68	4.79
two_second_target	128	128	32,768	4.32	1.67
no_overlap	128	128	32,768	3.96	4.56

These are independently optimized scenarios, not confidence intervals. The two-second-target row tightens the mean-latency cap to two seconds while retaining the central hardware assumptions. All other rows use five seconds. The CPU capacity assumptions are 0.5, 2, and 8 billion *weighted* local field operations per 64-vCPU node/s. They are not measured EC2 performance. Exchange-delay assumptions are 0.05, 0.2, and 0.5 ms; the central value is 0.2 ms. No-overlap sums CPU, network and RAM service time; other rows take their maximum. The latency simulation assumes Poisson arrivals and service-time coefficient of variation 0.05, and cannot predict real network tails or failures.



The central minimum uses 128 queries, fanout 128, 32,768 scalar labels in each flat base, and 3 recursive tree levels per cuckoo table. Its state occupies 1,926 GiB per party per replica. It uses 6 machines per party per replica and 530 independent replica groups (31,800 machines across all parties). Its modeled mean latency is 4.77 s, of which 1.29 s is dependent exchange time. This enormous deployment is an accounting result, not a recommended AWS launch. No EC2 capacity availability, quota increase, or linear scaling to this size has been established.

Level	Records per table	Payload words	Pool records	Lower path buckets
0	536,870,912	6	893	22
1	4,194,304	128	893	15
2	32,768	128	893	8

Per logical read, this point charges 3.36 million reduction outputs, 182.2 thousand preprocessed random bits, and 629.2 MB transmitted per party. The byte count includes the leading backend costs and 5%

framing. Stored-field-bit metadata reduces conversion work but expands persistent state to roughly 12–13 TiB per party for typical grid points. That is *not* the memory cost of a properly packed Boolean-sharing implementation; this model charges every bit as an authenticated arithmetic share.

Fanout 128 wins the central scenario because it removes recursive levels; the low-exchange scenario instead favors fanout 32. Thus neither "largest payload" nor "smallest base" is a robust optimum. For prototyping, test batches 128–1024, fanouts 16, 32, 64, and 128, and actual bases of a few thousand to 32768 labels. Small batches keep formation latency low; the model's several-second delay comes mainly from computation and dependent communication, not waiting to fill batches.

3 A cost envelope for optimization targets

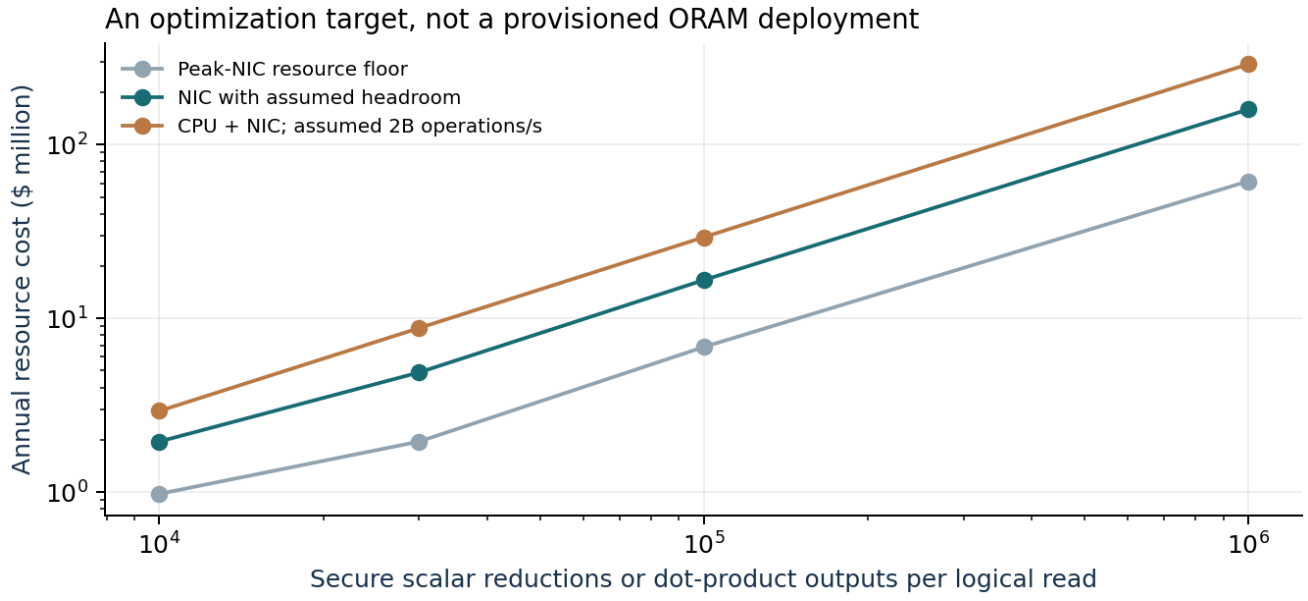
For G scalar reductions or private-dot-product *outputs* per read, the leading SY-Shamir traffic proxy is

$$V_{\text{party/read}} = G \cdot \frac{59}{6} \cdot 16 \text{ bytes}, \qquad V_{\text{party/sec}} = 10^4 V_{\text{party/read}}.$$

This excludes additional random bits, long-dot local work, conversions, routing, and special openings unless they are charged separately. It is a conditional backend relation, not an information-theoretic MPC lower bound.

Outputs/read	Peak NIC floor	NIC + headroom	Assumed CPU + NIC
	USD millions/year, all ten parties		
10,000	0.98	1.95	2.93
30,000	1.95	4.89	8.80
100,000	6.84	16.62	29.32
1,000,000	61.57	159.31	290.28

The table uses `r6in.32xlarge`, 200 Gbps, $11.15712/hour. Headroom includes 65% NIC utilization, 70% operating utilization, a 1.12 hottest-party factor, and 5% framing. CPU+NIC assumes 83 weighted operations/scalar output and 2 billion operations/s per 64 vCPU. It does not provision RAM, state replicas, dependent-round latency, intraparty routing, failover, or database initialization. Consequently even the rightmost column is not a sufficient deployment budget.



The gap from the reference's roughly 3 million outputs/read to a 30,000-output target is about 100-fold. Packing position maps alone cannot credibly be assigned that improvement. A substantially different metadata/routing implementation is needed before such a target can be presented as an expected system result.

4 Backend and scheduling assumptions

The accounting is tied to MP-SPDZ revision `73eb8556cdfc670c5d3dc1067a3ae5b4ad0d4441`, using authenticated SY-Shamir, $n = 10, t = 4, p = 2^{127} - 1$. The source-derived leading coefficient is approximately 59/6 field words sent per party per scalar/dot output; a checked opening costs 8 words, and an authenticated random bit about 24 words. A scalar multiplication has one exchange; the current dot-product path has two sequential exchanges. An active-check barrier adds at least four phases. Leading coefficients are not complete measured wall-clock costs.[1]

Keep these three kinds of arithmetic separate:

- One secret-selector payload mux uses one scalar product per field word.
- A k-bit secret comparison uses a prefix circuit of about $3k - 2$ products before its conditional movement; it is not one arithmetic gate.
- An inner product of length$k$ has one communicated output but$\Theta(k)$ local arithmetic. Random bits and exact field-to-bit conversion require their own preprocessing ledger.

The model's exact canonical-mask conversion and path-planning circuits have explicit proxy coefficients. The mask acceptance work is charged offline; conversion/opening phases are charged online. For a simple sustainable service schedule, the batch resource time includes *both* online and offline work. Ready preprocessing and overlapped maintenance can lower online latency, but their steady-state compute and network capacity must still be provisioned.

For K machines per party in one replica, the capacity model uses

$$T_{\text{service}} = D\tau + \max\{T_{\text{network}}, T_{\text{CPU}}, T_{\text{RAM}}\}, \qquad J = \left\lceil \frac{10^4 T_{\text{service}}}{0.7B} \right\rceil.$$

J is the number of independent evolving ORAM replicas. Each replica stores the entire shared database; K shards its physical state and computation. A 0.7 distributed-efficiency factor and explicit intraparty share-routing traffic limit ideal scaling. The model adds scratch and a rough traffic-derived preprocessing inventory proxy. An implementation must replace that proxy with actual material sizes. The price is $10JKp_{\text{hour}} \cdot 8760$.

A central dispatcher forms a batch before choosing a replica. Mean formation waiting is approximately $(B-1)/(2R)$: 6.35 ms at $B = 128$ and 51.15 ms at $B = 1024$. Splitting arrivals into$J$ independent batchers would multiply that delay and is not the deployment being priced. Every replica finishes maintenance before accepting its next batch. The script also simulates dispatch queues, and tests that inadequate service capacity causes queues to grow.

[1] Pinned protocol source: https://github.com/data61/MP-SPDZ/tree/73eb8556cdfc670c5d3dc1067a3ae5b4ad0d4441/Protocols.

5 AWS placement changes the answer

The official Linux/shared-tenancy US East (N. Virginia) offer snapshot is `20261012193925`, published 12 October 2026, with selected prices effective 1 October 2026. Raw rows and a refresh script are retained.[2]

Same physical AZ VPC-peering traffic can avoid transfer fees, including across accounts. Cross-AZ peering is$0.01/GB at each endpoint, or$0.02 for a transmitted GB counted once.[3] Thus at 10,000 outputs/read, the leading all-cross-AZ transfer bill alone is $99.2 million/year under this backend. For the central reference circuit it is $39.7 billion/year, before compute. A 4/3/3-party placement gives crossing fraction 66/90 only for uniformly weighted all-to-all communication; actual protocol traffic needs a byte-weighted graph. AWS billing-unit assumptions are exposed.

Advertised NIC bandwidth is per direction; single-flow limits and multi-NIC configuration matter. In particular, `r6in.32xlarge` needs both network cards to attain the aggregate 200 Gbps. No sustained MPC bandwidth or collective latency is guaranteed by the instance specification.[4] The prices exclude EBS, snapshots, setup, backups, client transfer, NAT, orchestration, failover reserve, support, taxes, and engineering. Same-provider or same-AZ placement does not by itself establish independent corruption domains.

6 Construction choice and what the estimates establish

The design target is a read service for $N = 2^{28}$ key–value pairs, each containing a 32-byte key and a 32-byte value, at 10,000 logical requests per second. The security target is ten parties with at most four malicious corruptions and security with abort. Mean response time should remain below five seconds, with lower latency preferred. The raw database is 16 GiB; cuckoo slack, tree buckets, shares, authentication, recursive maps, and independent service replicas add to this substantially. Logical reads still modify the ORAM's position maps and placement state. Application writes and their replication protocol are outside this estimate.

The proposed starting point is two-choice cuckoo hashing over recursive Circuit OPRAM, with both candidates probed on every lookup. Two separate cuckoo tables permit two independent OPRAMs to run in parallel; this modestly reduces individual pool sizes without requiring secret-dependent shard routing. The model uses single-slot hashing with explicit slack and a scanned overflow stash. Hash-table and ORAM stash capacities remain parameters that need a failure analysis. If clients supply secret shares of indices computed using public hash functions, the interface must specify which input consistency properties are checked. A keyed hash evaluated inside MPC is a different, additional cost; it has not been selected and is not implicitly free.

Circuit OPRAM is more than a batch of Circuit ORAM paths. For b probes into a binary tree with leaf depth H, let $r = \lceil \log_2(2b) \rceil$. The parallel construction keeps disjoint subtrees below depth r and represents the upper tree and stash in a pool. Its algorithms include private conflict resolution, pool retrieval, eviction candidate selection, and pool cleanup. The deterministic eviction paths occupy

[2] Official offer feed: https://pricing.us-east-1.amazonaws.com/offers/v1.0/aws/AmazonEC2/current/us-east-1/index.csv.

[3] https://docs.aws.amazon.com/vpc/latest/peering/what-is-vpc-peering.html; https://aws.amazon.com/vpc/pricing/.

[4] https://docs.aws.amazon.com/ec2/latest/UserGuide/ec2-instance-network-bandwidth.html; https://docs.aws.amazon.com/ec2/latest/instancetypes/mo.html.

distinct subtrees. These are essential costs, not an optional implementation detail.[5]

For accounting, the reference template uses

$$P = Z(2^r - 1) + R_{\text{stash}}, \qquad h = H - r + 1,$$

where Z is bucket capacity and R_{stash} is an exposed slack parameter. A conservative temporary allocation allows $P + 3b$ records while fetched blocks and returned eviction candidates are processed. The pool replaces the upper tree buckets; their storage must not be counted twice. Recursion stops before an additional tree would have fewer than $2b$ leaves. This capacity accounting does not certify the chosen concrete stash slack.

Reference compilation versus the best achievable system. The program constructs an explicit operation ledger for a plausible Circuit-OPRAM-inspired compilation. It includes pool operations, metadata conversion, record movement, duplicate handling, and eviction planning. Some scan and planning coefficients are deliberately exposed estimates. It is neither a complete circuit implementation nor a proven upper bound on every implementation detail. Large reduction counts therefore demonstrate the expense of this specified compilation; they are not a lower bound on MPC lookup, a proof that the throughput goal is infeasible, or a claim to have found the globally optimal construction. AWS prices translate these conditional workloads into dollars; they do not validate the workload or processor-throughput assumptions.

The accounting distinguishes scalar products, private inner-product outputs, local arithmetic, random bits, openings, memory traffic, and dependent communication phases. In an appropriate Shamir backend, an inner product can communicate per output rather than per scalar term. Its local computation still grows with its length. This distinction is particularly valuable for retrieval and flat position-map bases.

7 Optimizations with a concrete implementation path

Use private shuffles to reduce repeated payload movement. The optimized reference sort augments each sorting key with a secret unique ordinal, securely shuffles records, and then opens comparison outcomes. Subsequent sorting moves are public local operations. The unique ordinal is essential: otherwise duplicate-key comparison patterns can disclose multiplicity. Vectorizing comparisons across quicksort recursion levels exposes parallelism, although its height and latency remain random. This shuffle-then-sort approach is described in the Orq implementation. [6]

A generic realization has five fixed parties each choose a uniform private permutation and input checked switching-network controls. Under static corruption of at most four parties, at least one permutation remains hidden. Uniform switch bits alone would not produce uniform permutations. Restoring request order requires both undoing public sorting moves and applying the secret networks in reverse; the latter is charged. This is a useful reproducible baseline, but five sequential networks are expensive.

For a more ambitious implementation, benchmark a Shamir-specific permutation primitive. The 2026 Shuffle2 work uses inner products and preprocessing to obtain linear online shuffle communication in the malicious honest-majority setting. Its published implementation measures semi-honest execution; the paper's malicious overhead factors are estimates. Its offline cost also remains substantial. Consequently, it motivates an optimization experiment, not an unqualified replacement throughput assumption.[7]

[5] Chan and Shi, *Circuit OPRAM: Unifying Statistically and Computationally Secure ORAMs and OPRAMs*, https://eprint.iacr.org/2016/1084.

[6] *Orq: Complex Analytics on Private Data with Strong Security Guarantees*, SOSP 2025, https://cs-people.bu.edu/vkalavri/assets/pdf/orq-sosp25.pdf.

[7] Gao et al., *SLIDE: Shuffle Shamir Secret Shares Uniformly with Linear Online Communication and Guaranteed*

Specialize pool operations rather than sorting everything alike. Pool cleanup is stable compaction on validity bits. Local prefix sums and scalar products can compute a unique secret destination permutation, leaving private routing as the expensive primitive; long-key comparisons are unnecessary. Maintaining the pool in logical-address order also permits a derived finite-difference gather: attach differences of adjacent payloads to pool records, insert zero increments for requests, merge by address, and use local prefix sums to recover candidate values. Secret equality still decides whether an exact match exists. Candidate-selection routing must subsequently be undone and stable compaction must preserve the sorted-pool invariant. This optimization needs an implemented invariant check and security argument; it is not assumed by the reference scan coefficients.

Choose metadata representation separately from payload encoding. Moving one field word and comparing a secret integer have different costs. Repeatedly converting compact arithmetic metadata to bits is expensive; retaining every address and leaf bit as an arithmetic share expands both storage and routing. A practical comparison should include compact metadata, retained bit metadata, and a mixed representation with narrow comparisons and explicit conversion costs. Retained bits must be charged whenever their record moves. Leaf compatibility against a public eviction path can reuse prefix-match products; target-depth comparisons use roughly $\log_2 H$ bits, rather than the full key width.

The published depth analysis of Circuit ORAM provides a starting point for parallel eviction planning. Its word-depth bound is not directly an MPC-round bound: comparison, selection, and authentication phases remain to be compiled.[8]

Tune recursive packing and the base together. Packing F parent labels per recursive block reduces record counts but increases record width, private subblock selection, and routing cost. Moderate powers of two, for example $F = 8$ through 128, are useful search points rather than a predetermined optimum. The binary eviction tree can remain unchanged. Several requests may update different offsets of the same map block: group those updates privately and commit the block once. Independent whole-block writes would overwrite each other.

The base contains one scalar label per record in the last instantiated tree. It is not smaller by another factor of F unless another recursive tree has been paid for. With $n = 2^d$ base labels, a divide-and-conquer one-hot decoder has the derived multiplication recurrence

$$T(d) = T(\lfloor d/2 \rfloor) + T(\lceil d/2 \rceil) + 2^d, \qquad T(0) = T(1) = 0,$$

and depth $\lceil \log_2 d \rceil$. For $n = 4096$, this is 4,272 products per request and four product layers, before validity gating and read/write operations. After duplicate resolution, dot-product retrieval and aggregated updates need about $b + n$ reduction outputs, while local work remains proportional to bn. Compare this against a complete additional recursive level. The reference program uses the divide-and-conquer decoder, adding one product and a conservative extra phase for a secret representative-valid bit.

Optimize scheduling after retaining dependencies. At one level, pool fetch and subtree fetch can overlap once old labels are available. Maintenance on an already accessed recursive tree can overlap fetching from the next tree if remapping labels have been sampled beforehand and state updates are coordinated. Recursion still has a base-to-data fetch dependency, and the next batch must not observe an uncommitted state. A sum of all phase counts is a conservative serial schedule; replacing it by the maximum level depth would be unjustified. Centrally forming batches before dispatching them to

Output Delivery, especially Sections 5 and Appendix C.4, https://arxiv.org/html/2608.27129v1. Implementation: https://github.com/GJCPP/ShamirShuffle.

[8] Chan, Chung, and Shi, *On the Depth of Oblivious Parallel RAM*, https://eprint.iacr.org/2017/861.

independent replicas avoids multiplying formation delay by the number of replicas. At 10,000 requests/s, batch size Q alone adds approximately $(Q-1)/(2 \cdot 10{,}000)$ seconds of mean formation delay.

8 Validation needed before using a deployment budget

The next experiment should implement and measure one complete recursive batch, including duplicates and misses, on the selected malicious backend. Separately measure comparison/conversion, shuffle and inverse routing, pool cleanup, subblock updates, eviction, and preprocessing replenishment. Then measure the composed dependency schedule and memory footprint, rather than inferring end-to-end performance from multiplication throughput alone.

Failure parameters must cover the intended deployment lifetime. One year at 10,000 requests/s contains about $2^{38.2}$ logical requests, before counting two probes, recursive levels, replicas, and checking events. If A events each have failure probability at most ϵ, a simple lifetime bound is $A\epsilon$. For example, an annual target of 2^{-40} with 2^{42} relevant events would require per-event probability at most 2^{-82}. The ORAM stash, cuckoo overflow, statistical checks, and any randomized sorting limits need their own justified bounds. A simulated absence of overflow is useful testing evidence but does not establish such a bound.

Finally, capacity should include authentication and preprocessing, local routing between machines of one party, batch queues, and the full memory of each independently evolving replica. Cross-availability-zone placement changes both communication charges and latency. The resulting estimate is a proposal-planning scenario until an end-to-end malicious implementation has been calibrated on the intended topology.

9 Bottleneck clarification and scan comparison (13 October update)

The current model is limited first by total CPU demand, then by online communication. Storage is large, but is not the binding constraint at the selected point. This is a diagnosis of the analytical model, not a measured profile of an implemented MPC service. The earlier report combined online and preprocessing work; the new ledger separates them without changing its total costs.

For the selected 128-query batch on six `r6in.24xlarge` machines per party:

Resource or dependency	Time per batch
CPU, including preprocessing	3.452 s
Network, including preprocessing and internal routing	2.225 s
Modeled RAM traffic	0.094 s
Dependent exchanges	1.291 s
Service time: $1.291 + \max(3.452, 2.225, 0.094)$	4.743 s

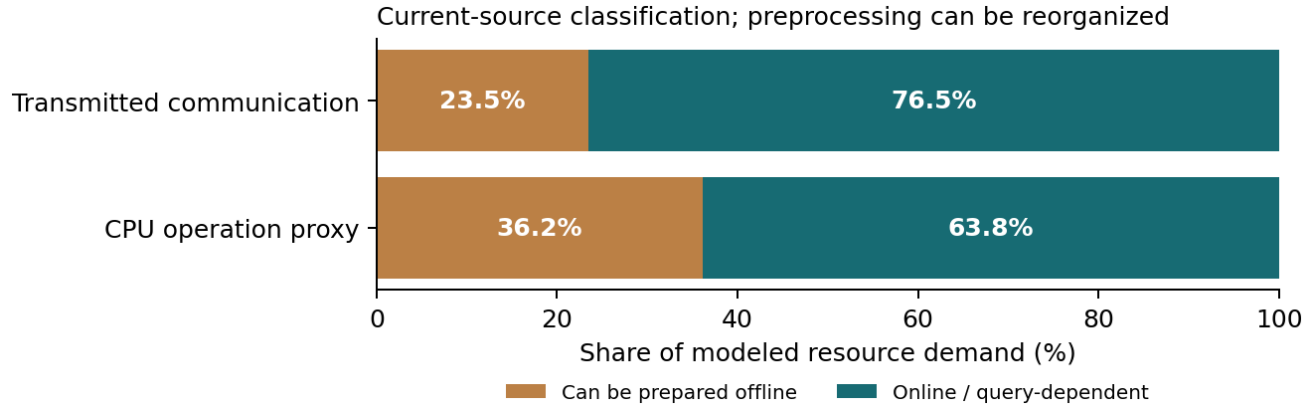
These resource times are not additive under the model's overlap assumption. Total mean response time is about 4.77 s after batch formation and fixed response overhead. Faster CPUs eventually make networking or round latency dominant.

9.1 What is offline and what is online?

The selected SY-Shamir backend does not consume one Beaver triple per scalar multiplication. Its main product/degree-reduction traffic depends on the live inputs. Preprocessing supplies fresh checking randomness, random bits, and canonical masks. Thus importing a triple-generation bottleneck from a different SPDZ backend would misdiagnose this model.

Per party, per logical read	Preprocessing	Online	Total
Sent traffic, decimal MB	148.0	481.1	629.2
CPU proxy, million weighted operations	122.9	216.9	339.8

The traffic includes 5% framing, counts each send once, and excludes intraparty redistribution. CPU units are weighted source-operation estimates, not measured cycles or seconds. All ten parties incur the corresponding resource demand.



For O scalar/dot outputs, C canonical masks, and B_r random bits, the leading preprocessing allocation is

$$W_{\text{off}} = \tfrac{5}{6}O + 9(126C) + 24B_r + 8C,$$
$$A_{\text{off}} = 15O + 68(126C) + 330B_r + 35C.$$

W counts transmitted field words per party; A is the local CPU proxy. A full output costs $9 + 5/6$ words: 9 for query-dependent resharing and 5/6 for a fresh secret checking coefficient. Mask acceptance uses 126 outputs and an opening offline. Fixed checks add small terms retained in the program. The online column is the unchanged total minus this allocation.[9]

The CPU phase boundary can move. Precomputing random polynomial contributions to scalar resharing can shift another 48 operations per live scalar output out of the online loop. In this sensitivity, preprocessing rises from 36.2% to about 81.1% of total CPU work, with unchanged steady-state arithmetic. This requires additional material storage, memory traffic, and a correct scheduling implementation. It does not make that computation or its dollar cost disappear. The 9-word live communication remains query-dependent. Likewise, ready preprocessing can improve response time even when the service's sustainable throughput is still limited by generating it. The existing 4.74 s service estimate charges full resource production per batch; it is not an online-only latency benchmark.

9.2 Storage and replication

The selected state is 1,926 GiB (1.88 TiB) per party per ORAM replica, or about 2,217 GiB including the modeled scratch/reserve. At 75% usable memory, each 768 GiB instance provides 576 GiB, so four machines suffice for capacity. Six are selected to meet the latency/throughput model. Removing the memory constraint and re-searching the same instance size leaves the selected price unchanged. Storage would become binding under another representation or scheduling policy; field-bit metadata in particular expands it substantially.

The previous $2.33B/year deployment dedicates machines to each of 530 independently evolving ORAM states. An asynchronous scheduler could share machines across those states during round waits. The aggregate CPU-capacity requirement is 2,312 machines per party (about $1.69B/year), and their aggregate RAM could hold those 530 states. That is a resource-capacity estimate, not proof that a scheduler achieves the latency bound. The extra dedicated-instance cost is not an inherent cryptographic lower bound. Do not allocate VM dollars proportionally to the CPU/traffic percentages: one fleet serves overlapping demands.

9.3 Naive parallel full-key scan

The direct baseline compares every query key with all $N = 2^{28}$ stored keys, then returns $\sum_i [k_i = q]v_i$. Comparisons run in parallel across rows and queries; rows are visited in a public order. Values use three private dot products, so their selection communicates per output, not per record. There is no ORAM maintenance. The dictionary has unique keys; the sum of match bits gives a private hit indicator.

The baseline uses *exact* equality optimized for this field. Represent a key with the same three field limbs as the ORAM. Since $p = 2^{127} - 1 \equiv 3 \pmod 4$, -1 is a quadratic nonresidue and $a^2 + b^2 = 0$ holds exactly when both inputs are zero. For limb differences d_0, d_1, d_2, compute

$$u = d_0^2 + d_1^2, \qquad v = u^2 + d_2^2, \qquad e = 1 - v^{p-1}.$$

[9] Source accounting is pinned to MP-SPDZ commit `73eb8556cdfc670c5d3dc1067a3ae5b4ad0d4441`; see https://github.com/data61/MP-SPDZ/tree/73eb8556cdfc670c5d3dc1067a3ae5b4ad0d4441/Protocols and the local ledger `data/mpc_sources.txt`. These are derived counts, not an official benchmark table.

Each sum of squares is a length-two private dot product with one reduced output; the 135-product exponentiation is the addition-chain routine from the model. Thus each comparison costs 137 secure outputs, versus 407 for independently zero-testing and combining three limbs. This equality also improves the ORAM's few final key checks, but changes its millions-of-outputs total by less than 0.1%. Intermediate dot results must be degree-reduced before reuse.

Ignoring the tiny shared frontend, the full scan requires

$$G_{\text{scan}} = 137N + 3 \simeq 36.8 \text{ billion outputs/read.}$$

Batching does *not* divide this count by batch size: a batch has $137NB$ comparison outputs. It reduces communication scheduling overhead and permits database-cache reuse. Use row/query tiling, not an $N \times B$ materialized equality matrix. The read-only 48 GiB of authenticated database shares per party can be shared across concurrent workers, unlike independent mutable ORAM placements.

9.4 An indexed alternative using private dot products

An indexed scan is another baseline: keep the same two cuckoo tables and scan their arrays without ORAM. Private dot products allow a two-dimensional read. Arrange each $D = 2^{29}$-cell table as an $A \times C$ matrix, with $AC = D$. For private row and column one-hot vectors a, b, compute, for each payload word,

$$z_i = \sum_j b_j X_{ij}, \qquad y = \sum_i a_i z_i.$$

This reads every cell through local arithmetic, but only $AW + W$ dot outputs are reduced. With $W = 6$ key/value words, communication is minimized near $A = \sqrt{D/(W+1)}$. The model chooses $A = 8192, C = 65536$. One-hot generation is charged, both tables are always probed, and full keys plus the cuckoo stash are checked exactly. Empty-cell validity can use an unused bit in the last key limb, so the six-word payload need not grow. This is a derived exact read construction; it does not imply a fast write protocol.

Its approximately 249,000 outputs/read are *fewer* than the ORAM reference, but its dot products still process both full arrays. The current source proxy charges approximately $4 \cdot 2DW = 25.77$ billion local operations/read before the small remaining costs. Packing the batch as matrix multiplication can change the actual cycles/operation; this must be measured independently. Our CPU-rate assumption is not a benchmark of that dense kernel. Both scan variants can pipeline queries and reuse immutable shares without paying for one full state per worker.

9.5 Side-by-side work and cost

Per logical read / party	OPRAM	Full-key scan	Indexed 2D scan
Secure outputs	3.36M	36.78B	0.249M
Local CPU proxy	0.340B	3,059B	25.79B
Sent bytes, including framing	629MB	6.08TB	41.2MB
Persistent shares	1,926GiB	48GiB	192GiB
Exchange-depth contribution	1.291s	0.030s	0.031s

Storage is per complete state per party, not per read. Exchange-depth figures are only the phase-delay term at 0.2 ms/exchange; they are not achievable response latencies without sufficient computation and networking. Memory traffic is optimistically tiled/reused for scans; detailed scratch and reduction aggregation are not claimed to be free in an implementation.

Under the *same unmeasured* 2-billion-weighted-operations/s per 64-vCPU assumption, the common aggregate resource-capacity calculation gives:

Construction	Annual resource equivalent, ten parties
OPRAM reference	$1.69 billion
Naive full-key scan	$15.25 trillion
Indexed 2D dot scan	$128.6 billion

These all use `r6in.24xlarge` at the same 12 October price snapshot, 70% distributed efficiency and 70% utilization. They are not new deployment quotes. The scan fleets are astronomically large extrapolations, not plausible AWS reservations. The common calculation excludes detailed sharded-dot aggregation, queue/SLA feasibility, setup and scratch provisioning; its purpose is to expose the scale of work. The previous $2.33B ORAM figure includes a particular dedicated replica policy and should not be compared with a CPU-only scan calculation as if both were equally complete deployments.

Relative to the OPRAM reference, the naive scan needs about 9,000 times the CPU proxy and 9,656 times the traffic. The stronger indexed scan needs about 76 times the CPU proxy but only 6.5% of the traffic. Therefore "scan is communication-heavy" is true for the naive full-key implementation, but false as a universal statement.

Practical conclusion. The next measurements should isolate (1) total preprocessing production, (2) live product/resharing throughput, (3) checked network exchanges and (4) a batched local matrix-contraction kernel. The current arithmetic OPRAM reference uses less modeled CPU work than these scans, while the two-dimensional scan is a serious bandwidth-oriented comparator. Neither the numerical CPU split nor the cheapest real system is settled without those measurements.

Reproduce the comparison with `python -m mpc_cost.compare_scans`; the complete ledger is `results/bottleneck_scan_comparison.json`.

10 Reproduction and next decision

From the repository root:

```
python -m mpc_cost.run --full
python -m mpc_cost.compare_scans
python -m unittest discover -s mpc_cost -p "test_*.py" -v
python -m mpc_cost.render_report
```

Compile `mpc_cost/mpc_oram_cost_report.tex` with its working directory set to `mpc_cost`. The program has no runtime dependencies beyond Python's standard library; generating figures additionally needs Matplotlib and PDF compilation needs LaTeX. `README.txt` explains overrides and price refresh.

The tests validate arithmetic witnesses (comparison, gated decoding, duplicate updates, Fermat equality), recursive base dimensions, metadata movement, all-party and replica billing, transfer send-counting, and queue saturation. They do not validate an unimplemented ORAM security proof or MPC performance.

Before choosing a proposal budget, benchmark checked scalar batches, long dot products, mask production, and a complete recursive read with duplicate updates. Measure online latency and sustainable total resource use separately, then replace the proxy counts and sweep the parameters again.